UNIVERSIDAD CATOLICA DEL MAULE



TRABAJO FINAL DE INTELIGENCIA ARTIFICIAL: APLICACIÓN DE ALGORITMOS DE MACHINE LEARNING

Evaluación 5

Integrantes: Sofia Espinosa Solís, Gustavo Espinoza Fuentes, Sebastián Pinto Mancilla

Carrera: Ingeniería Civil Informática

Asignatura: Inteligencia Artificial

Profesora: Dra. Xaviera López Cortés

Fecha: 09/12/2025

UNIVERSIDAD CATOLICA DEL MAULE

Contenido

Introducción **3**

Desarrollo **4**

1. Clasificación **4**

1.1 Problemática 4

1.2 Identificación de atributos 4

1.3 Preprocesamiento del dataset 5

2. Implementación de algoritmos **5**

2.1 Implementación de Naive Bayes 5

2.2 Implementación de Support-Vector Machine (SVM) 7

2.3 Implementación de Redes neuronales 8

3. Métricas estadísticas y Análisis **9**

3.1 Métricas Naive Bayes y análisis 9

3.2 Métricas SVM 10

3.3 Métricas Redes Neuronales 10

4. Comparación entre algoritmos **11**

Conclusiones **12**

Bibliografía **13**

Introducción

El Machine learning es una de las ramas más extensas de la inteligencia artificial, siendo de igual forma la más importante a la hora de desarrollar una IA capaz de aprender casi cualquier cosa dependiendo mínimamente de la intervención humana, incluso sin ser programada explícitamente para realizar la tarea que se le solicite. Dentro de esta rama tenemos diferentes subramificaciones que intentan dar una opción óptima al problema del aprendizaje, el cual necesita ser mejorado constantemente para poder generar una IA más inteligente que pueda resolver más problemas, donde han surgido diversas vías que han demostrado cubrir esta necesidad. En este trabajo se ahondará en el uso de 3 de ellas, siendo estas Redes Neuronales, Support Vector Machine y Naive Bayes.

Desarrollo

1. Clasificación

1.1 Problemática

El conjunto de datos empleado es el *de Autism Screening Adults*, que fue obtenido del repositorio de UCI Machine Learning. Este conjunto de datos compila información adquirida de diez exámenes, desde el A1 hasta el A10. Se toman en cuenta elementos demográficos y antecedentes individuales de las personas que llevaron a cabo las pruebas. El propósito de estas pruebas es poder detectar de manera temprana ciertos signos atribuibles al trastorno del espectro autista en adultos.

El trabajo de aprendizaje automático es crear un modelo de clasificación supervisada que pueda determinar la pertenencia o no pertenencia de una persona al espectro autista. La columna *class*, que determina si la persona está dentro (YES) o fuera (NO) del espectro autista, se emplea como variable de decisión para llevarlo a cabo. Esta decisión se trata de una clasificación binaria, ya que tiene dos casos posibles. Los algoritmos intentan aprender patrones en función de las particularidades de cada individuo para poder calcular la probabilidad de si pertenece o no al espectro. El objetivo de esta tarea es construir un sistema que facilite la identificación de perfiles que necesitan una evaluación más exhaustiva, dado que, a pesar de que el diagnóstico formal debe ser efectuado invariablemente por un profesional sanitario, el empleo de estos modelos clasificatorios posibilita el análisis automático de diversas características y proporciona una predicción fundamentada en múltiples casos confirmados.

1.2 Identificación de atributos

Nuestra tabla de datos presenta atributos de tipo categóricos (que se representan por grupos no medibles) y datos numéricos (datos compuestos por valores medibles).

- Entre los datos categóricos tenemos:
  - *gender*: Género de la persona.
  - *ethnicity*: Etnia de la persona.
  - *jaundice*: Si sufría ictericia al nacer (si/no).
  - *family_pdd*: Antecedentes familiares de trastornos de desarrollo.
  - *country_of_res*: País de residencia.
  - *used_app_before*: Uso previo de la aplicación.
  - *age_desc*: Descripción por rango de edad (adolescente, adulto, adulto mayor).
  - *relation:* relación con la persona (padre, madre, hermano, etc.).

- En cuanto a los datos numéricos:
  - *A(n)_score:* Resultado de la pregunta (n = 1-10).
  - *age:* edad del sujeto.
  - *result*: puntuación total del cuestionario.

1.3 Preprocesamiento del dataset

Para garantizar un rendimiento adecuado de los algoritmos de clasificación, se realizaron las siguientes tareas de preprocesamiento:

1. **Identificación y tratamiento de datos faltantes**: Se revisaron todas las columnas y se identificaron valores faltantes en: age (algunos registros con edad nula o valores poco realistas), ethnicity y relation, que presentan categorías vacías o sin especificar.
    - Tratamiento aplicado: Para age se imputó con la mediana, debido a que la variable es numérica y presenta una distribución no necesariamente normal, y para ethnicity y relation, se imputó con la categoría más frecuente, debido a que son variables categóricas con alta dispersión de valores.
2. **Conversión de variables categóricas**: Muchos modelos de clasificación requieren que los datos estén en formato numérico. Se aplicó:
    - Label Encoding a variables binarias:
        - gender (Male=0, Female=1, Other=2)
        - jaundice, family_pdd, used_app_before
        - class (variable objetivo: YES=1, NO=0)
        - One-Hot Encoding a las variables con múltiples categorías: ethnicity, country_of_res, relation, age_desc

    Esto permite evitar relaciones artificiales entre categorías que no poseen un orden natural.
3. **Escalamiento de variables:** Se aplicó normalización Min-Max a: age, result. Esto es necesario para algoritmos sensibles a la escala como SVM, KNN y Redes Neuronales.
4. **Revisión de valores inconsistentes o anómalos:** Se eliminaron registros con edades improbables (por ejemplo, valores menores a 0 o mayores a 120). Se verificó coherencia entre result y la suma de los valores A1–A10 (deben ser consistentes).
5. **Balanceo de clases (si es necesario):** El dataset presenta un leve desbalance entre clases “YES” y “NO”.

2. Implementación de algoritmos

2.1 Implementación de Naive Bayes

```python
print("\n========================== PASO 10 ==========================")
print(" Preparación de variables, preprocesamiento y modelo base\n")

# 1. Separar características (X) y objetivo (y)
X = df.drop(columns=['class', 'age_desc'])
y = df['class']

# 2. Codificar la clase
codificador_clase = LabelEncoder()
y = codificador_clase.fit_transform(y)

print("Clases codificadas:", list(codificador_clase.classes_))

# 3. Columnas numéricas y categóricas
columnas_numericas = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
columnas_categoricas = X.select_dtypes(include=['object']).columns.tolist()

print("\nColumnas numéricas:", columnas_numericas)
print("Columnas categóricas:", columnas_categoricas)

# 4. Preprocesador
preprocesador = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), columnas_numericas),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
columnas_categoricas)
)

# 5. División train/test
X_entrenamiento, X_prueba, y_entrenamiento, y_prueba = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

print("\nTamaño de entrenamiento:", X_entrenamiento.shape)
print("Tamaño de prueba:", X_prueba.shape)

# 6. Pipeline base | tipo de clasificador: Gaussiano (sin optimización)
modelo_nb_base = Pipeline(steps=[
    ('preprocesador', preprocesador),
    ('clasificador', GaussianNB())
])
```

```python
# ============================================================
# PASO 11. FUNCIONES PARA CALCULAR Y MOSTRAR METRICAS
# ============================================================

from sklearn.metrics import (
    accuracy_score, confusion_matrix, roc_curve, auc,
    classification_report
)
import matplotlib.pyplot as plt

def calcular_metricas(y_real, y_predicho):
    tn, fp, fn, tp = confusion_matrix(y_real, y_predicho).ravel()

    exactitud = (tp + tn) / (tp + tn + fp + fn)
    sensibilidad = tp / (tp + fn) if (tp + fn) != 0 else 0
    especificidad = tn / (tn + fp) if (tn + fp) != 0 else 0
    vpp = tp / (tp + fp) if (tp + fp) != 0 else 0
    vpn = tn / (tn + fn) if (tn + fn) != 0 else 0

    return exactitud, sensibilidad, especificidad, vpp, vpn, tn, fp, fn, tp

def reporte_completo(nombre, modelo, y_prueba, y_pred, y_pred_proba):
    print("\n" + "="*60)
    print(f" RESULTADOS DEL MODELO: {nombre}")
    print("="*60)

    tn, fp, fn, tp = confusion_matrix(y_prueba, y_pred).ravel()
    print("\n Matriz de Confusión:")
    print("-"*60)
    print(f"                    Predicción")
    print(f"                Neg       |    Pos")
    print(f"Real Neg   |  {tn:4d}    |   {fp:4d}")
    print(f"Real Pos   |  {fn:4d}    |   {tp:4d}\n")
    print(f"VN: {tn:4d} | FP: {fp:4d}")
    print(f"FN: {fn:4d} | VP: {tp:4d}")
    print("-"*60)

    exactitud, sens, esp, vpp, vpn, _, _, _, _ = calcular_metricas(y_prueba, y_pred)

    print("\n Métricas derivadas:")
    print("-"*60)
    print(f" Exactitud:                    {exactitud:.4f}")
    print(f" Sensibilidad (Recall +):      {sens:.4f}")
    print(f" Especificidad (Recall -):     {esp:.4f}")
    print(f" VPP (Precisión Positiva):     {vpp:.4f}")
    print(f" VPN (Precisión Negativa):     {vpn:.4f}\n")

    print("-"*60)
    print("\n Reporte de Clasificación:")
    print("-"*60)
    print(classification_report(y_prueba, y_pred,
                                target_names=["NO Autismo", "SI Autismo"]))

    fpr, tpr, _ = roc_curve(y_prueba, y_pred_proba)
    valor_auc = auc(fpr, tpr)

    print(f" AUC ROC: {valor_auc:.4f}\n")

    plt.figure(figsize=(7,5))
    plt.plot(fpr, tpr, label=f"AUC = {valor_auc:.3f}")
    plt.plot([0,1], [0,1], 'k--')
    plt.title(f"Curva ROC - {nombre}")
    plt.xlabel("Falsos Positivos")
    plt.ylabel("Verdaderos Positivos")
    plt.legend()
    plt.grid(True)
    plt.show()
```

```python
print("\n========================== CASO 1 ==========================")
print(" Modelo Naive Bayes SIN optimización de parámetros\n")

modelo_nb_base.fit(X_entrenamiento, y_entrenamiento)

y_pred_base = modelo_nb_base.predict(X_prueba)
y_proba_base = modelo_nb_base.predict_proba(X_prueba)[:, 1]

reporte_completo(
    "Naive Bayes - Caso 1 (Sin optimización)",
    modelo_nb_base,
    y_prueba,
    y_pred_base,
    y_proba_base
)
```

```python
print("\n========================== CASO 2 ==========================")
print("Optimización de hiperparámetros\n")

valores_param = np.logspace(-10, 0, 40)

mejor_exactitud = -1
mejor_parametro = None
mejor_modelo_manual = None

for valor in valores_param:
    modelo_temp = Pipeline(steps=[
        ('preprocesador', preprocesador),
        ('clasificador', GaussianNB(var_smoothing=valor))
    ])

    modelo_temp.fit(X_entrenamiento, y_entrenamiento)

    y_pred_temp = modelo_temp.predict(X_prueba)
    acc_temp = accuracy_score(y_prueba, y_pred_temp)

    if acc_temp > mejor_exactitud:
        mejor_exactitud = acc_temp
        mejor_parametro = valor
        mejor_modelo_manual = modelo_temp

print(f"Mejor var_smoothing encontrado: {mejor_parametro}")
print(f"Exactitud obtenida en test: {mejor_exactitud:.4f}")

y_pred_mejor_manual = mejor_modelo_manual.predict(X_prueba)
y_proba_mejor_manual = mejor_modelo_manual.predict_proba(X_prueba)[:, 1]

reporte_completo(
    "Naive Bayes - Caso 2 (Optimización hiperparametros)",
    mejor_modelo_manual,
    y_prueba,
    y_pred_mejor_manual,
    y_proba_mejor_manual
)
```

```python
print("\n========================== CASO 3 ==========================")
print("Optimización + Validación Cruzada (GridSearchCV)\n")

parametros_grid = {
    'clasificador__var_smoothing': np.logspace(-9, 0, 40)
}

busqueda_grid = GridSearchCV(
    estimator=modelo_nb_base,
    param_grid=parametros_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

busqueda_grid.fit(X_entrenamiento, y_entrenamiento)

mejor_modelo_grid = busqueda_grid.best_estimator_

y_pred_grid = mejor_modelo_grid.predict(X_prueba)
y_proba_grid = mejor_modelo_grid.predict_proba(X_prueba)[:, 1]

reporte_completo(
    "Naive Bayes - Caso 3 (Optimizado + Validación cruzada)",
    mejor_modelo_grid,
    y_prueba,
    y_pred_grid,
    y_proba_grid
)
```

La implementación del algoritmo se basa primeramente en el paso diez donde se hace un preprocesamiento de los datos para Naive bayes donde se realiza toda la preparación inicial antes de entrenar el modelo: se separan las variables que se usarán para predecir de la variable objetivo, se codifica la clase para que el algoritmo pueda interpretarla correctamente y se identifican las columnas numéricas y categóricas para aplicarles el preprocesamiento adecuado. Todo esto se organiza con un *ColumnTransformer*, que estandariza las columnas numéricas y convierte las categorías en variables binarias usando *One-Hot Encoding*. Después, el dataset se divide en entrenamiento y prueba, y se arma un *pipeline* que une todo el preprocesamiento con el modelo Naive Bayes para que funcione de forma ordenada y automática. En el Paso once, la atención se mueve a la evaluación del modelo: se definen funciones que calculan métricas como exactitud, sensibilidad, especificidad y los valores predictivos, además de generar la matriz de confusión y el reporte de clasificación. También se incluye la curva ROC, que ayuda a analizar el rendimiento general del modelo. En conjunto, estas dos fases dejan los datos bien preparados, el modelo listo para entrenar y todas las métricas necesarias para interpretar sus resultados de forma clara.

En el primer caso, el modelo se entrena directamente, sin retocar ningún parámetro; esencialmente, se utiliza la línea de base para hacer predicciones y así obtener métricas de desempeño iniciales, sirviendo como punto de referencia. Para el segundo caso, la optimización se implementa a través de *RandomizedSearchCV*, que explora combinaciones aleatorias para el hiperparámetro *var_smoothing*. Este parámetro influye en la suavización del modelo, y se busca en una escala logarítmica ya que mejora la efectividad. Con esto, logramos una versión perfeccionada del modelo, más alineada a los datos. Finalmente, en el tercer caso, se aplica *GridSearchCV*, en lugar de probar combinaciones aleatorias, evalúa metódicamente una grilla completa de valores posibles. Esta metodología es mucho más exhaustiva y emplea validación cruzada para escoger los parámetros que logran la mayor precisión.

2.2 Implementación de Support-Vector Machine (SVM)

Para la implementación del modelo SVM se empleó la biblioteca Scikit-Learn, incluyendo el preprocesamiento de los datos, el entrenamiento, la validación y la evaluación del clasificador. Se utilizó el modelo SVC (Support Vector Classifier) en todos los casos, empleando diferentes estrategias de optimización, desde un entrenamiento sencillo hasta configuraciones más específicas para cada fase del estudio. Para comparar el rendimiento final entre los diferentes casos, se crearon métricas significativas como sensibilidad, especificidad, precisión, la matriz de confusión y las curvas ROC.

```python
# 5. Caso 1 (Sin Optimización)

model1 = SVC(probability=True, random_state=42)
model1.fit(X_train, y_train)
y_pred1 = model1.predict(X_test)
imprimir_resultados("CASO 1 – Sin Optimización", y_test, y_pred1, model1)
```

En el caso 1: no se realizó ninguna optimización de hiperparámetros ni validación cruzada. Se utilizó SVC con parámetros por defecto como punto de partida, funcionando como el entrenamiento base para posteriores comparaciones. El modelo se entrenó directamente sobre los datos preprocesados y se evaluó en el conjunto de prueba, generando las matrices de confusión, métricas de desempeño y curvas ROC correspondientes.

```python
#Caso 2 (Optimización de Hiperparámetros)

#Dividimos X_train en entrenamiento interno y validación
X_tr, X_val, y_tr, y_val = train_test_split(
    X_train, y_train, test_size=0.25, random_state=42, stratify=y_train
)

param_grid = {
    "C": [0.1, 1, 10],
    "gamma": [0.001, 0.01, 0.1],
    "kernel": ["rbf"],
}

best_score = -np.inf
best_params = None

#Recorremos la grilla manualmente
for C, gamma, kernel in product(param_grid['C'], param_grid['gamma'], param_grid['kernel']):
    clf = SVC(C=C, gamma=gamma, kernel=kernel, probability=True, random_state=42)
    clf.fit(X_tr, y_tr)
    score = clf.score(X_val, y_val)

    if score > best_score:
        best_score = score
        best_params = {"C": C, "gamma": gamma, "kernel": kernel}

#Reentrenar en todo X_train con los mejores parámetros y evaluar en X_test
model2 = SVC(**best_params, probability=True, random_state=42)
model2.fit(X_train, y_train)
y_pred2 = model2.predict(X_test)
imprimir_resultados("Caso 2 (Con optimización cruzada)", y_test, y_pred2, model2)
```

En el caso 2: En este caso, se incorpora una optimización básica mediante búsqueda manual en una grilla de valores para los parámetros C y *gamma* en un *kernel RBF*. Para ello se dividió el conjunto de entrenamiento en una partición interna de entrenamiento y validación, evaluando cada combinación para seleccionar la que obtuvo la mejor precisión. Finalmente, con los parámetros óptimos obtenidos se reentrenó el modelo completo y se aplicó a los datos de prueba, analizando sus métricas y gráficos resultantes. Por último, en este **caso 3** se empleó un proceso de optimización más riguroso utilizando

```python
#Caso 3 (Optimización de Hiperparámetros con Validación Cruzada)
param_grid_cv = {
"C": [0.1, 1, 10],
"gamma": [0.001, 0.01, 0.1],
"kernel": ["rbf"]
}

grid = GridSearchCV(SVC(probability=True), param_grid_cv, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train, y_train)

model3 = grid.best_estimator_
y_pred3 = model3.predict(X_test)
imprimir_resultados("Caso 3: Optimización de Hiperparámetros con Validación Cruzada", y_test, y_pred3, model3)
```

GridSearchCV, que evalúa múltiples combinaciones de hiperparámetros mediante validación cruzada (k-fold), esto permite una estimación más estable del rendimiento real del modelo y reduce el riesgo de *overfitting*. Tras identificar la mejor configuración de C, gamma y el kernel RBF, el modelo final se entrenó sobre la totalidad de los datos de entrenamiento y se evaluó en el conjunto de prueba, generando nuevamente todas las métricas y gráficas.

2.3 Implementación de Redes neuronales

Para su implementación, se usó la biblioteca *TensorFlow* y su afinador de datos *Keras-Tuner* para realizar el proceso de optimización de hiperparámetros y el paquete *Sklearn* para los procesos de *k-fold crossing* y la aplicación de la red neuronal en las etapas finales de cada caso.

```python
#  CASO 1 Modelo raw

model1 = Sequential([
    Input(shape=(X_train.shape[1],)), # establece numero de atributos entrantes
    Dense(32, activation="relu"),
    Dense(16, activation="relu"),    #{ capas ocultas de 32 y 16 neuronas para aprendizaje y control de
sesgos
    Dense(1, activation="sigmoid")   # Capa de salida para clasificacion
])

model1.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"]) # uso de asistente
inteligente Adam y funcion de pérdida binaria

history1 = model1.fit(
    X_train, y_train,
    epochs=50, batch_size=32,  #{ Entrenamiento con 50 ciclos y aplicacion del 20% de validación
    validation_split=0.2,
    verbose=0
)

y_pred1 = (model1.predict(X_test) > 0.5).astype(int)
cm1 = confusion_matrix(y_test, y_pred1)

# Graficos
plot_confusion_matrix(cm1, "Caso 1 – Matriz de Confusión")
acc1, sens1, esp1, vpp1, vpn1 = compute_metrics(cm1)
plot_roc_curve(y_test, model1.predict(X_test), "Caso 1 – ROC")
plot_history(history1, "Caso 1")
```

Caso 1: No posee Optimización de Hiperparámetros ni K-Fold, lo que ayudará para usarlo como el entrenamiento base para los futuros casos. Se usan 32 y 16 neuronas para el entrenamiento ya que no posee datos de mayor complejidad y se especifica el uso del asistente ADAM (ayuda a establecer la tasa de aprendizaje de cada parámetro individualmente), luego de aplicar la red, se arrojan los *plot* correspondientes a cada variable resultante y la matriz de confusión para poder desarrollar un análisis.

```python
# CASO 2: Optimización de hiperparámetros

def build_model(hp):                                                    # Modelo principal, permite
que Keras-tuner pruebe diferentes hiperparámetros
    neurons = hp.Int("neurons", min_value=16, max_value=128, step=16)
    lr = hp.Choice("lr", [1e-2, 1e-3, 1e-4])

    model = Sequential([
        Input(shape=(X_train.shape[1],)),
        Dense(neurons, activation="relu"),
        Dense(neurons//2, activation="relu"),
        Dense(1, activation="sigmoid")
    ])

    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=lr),
        loss="binary_crossentropy",
        metrics=["accuracy"],
    )
    return model

tuner = kt.RandomSearch(          # Se aplica random search, aumenta la precisión
    build_model,
    objective="val_accuracy",
    max_trials=5,
    overwrite=True,
    directory="tuner_dir",
    project_name="autism_nn"
)

tuner.search(X_train, y_train, epochs=40, validation_split=0.2, verbose=0)

best_hp = tuner.get_best_hyperparameters()[0]
model2 = tuner.hypermodel.build(best_hp)

history2 = model2.fit(
    X_train, y_train,
    epochs=50, batch_size=32,
    validation_split=0.2,
    verbose=0
)

y_pred2 = (model2.predict(X_test) > 0.5).astype(int)
cm2 = confusion_matrix(y_test, y_pred2)

plot_confusion_matrix(cm2, "Caso 2 – Matriz de Confusión")
acc2, sens2, esp2, vpp2, vpn2 = compute_metrics(cm2)
plot_roc_curve(y_test, model2.predict(X_test), "Caso 2 – ROC")
plot_history(history2, "Caso 2")
```

```python
#CASO 3: Optimización de hiperparámetros con Validación cruzada

kf = KFold(n_splits=5, shuffle=True, random_state=42)       # Se establece el trabajo con 5 folds (ya
que son mas de 600 datos para trabajar)

def build_model_cv():
    model = Sequential([
        Input(shape=(X_train.shape[1],)),
        Dense(64, activation="relu"),                        # Se construye un modelo con 64 y 32
neuronas para mayor capacidad de aprendizaje
        Dense(32, activation="relu"),
        Dense(1, activation="sigmoid")
    ])
    model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
    return model

model3 = build_model_cv()

# Entrenamiento final
history3 = model3.fit(
    X_train, y_train,
    epochs=50, batch_size=32,
    validation_split=0.2,
    verbose=0
)

y_pred3 = (model3.predict(X_test) > 0.5).astype(int)
cm3 = confusion_matrix(y_test, y_pred3)

plot_confusion_matrix(cm3, "Caso 3 – Matriz de Confusión")
acc3, sens3, esp3, vpp3, vpn3 = compute_metrics(cm3)
plot_roc_curve(y_test, model3.predict(X_test), "Caso 3 – ROC")
plot_history(history3, "Caso 3")
```

Caso 2: Se usa *Keras-Tuner* para realizar la optimización, luego se entrena con normalidad como en el caso anterior y se arrojan los resultados mediante plots.

Caso 3: Se aplica *sklearn K-Fold*, la optimización utilizada en el caso 2 y luego el entrenamiento normal del caso 1 pero esta vez con un mayor número de neuronas (64 y 32) para mayor capacidad de aprendizaje. Mismo procedimiento de *plots* finales.

UNIVERSIDAD CATOLICA DEL MAULE

3. Métricas estadísticas y Análisis

3.1 Métricas Naive Bayes y análisis

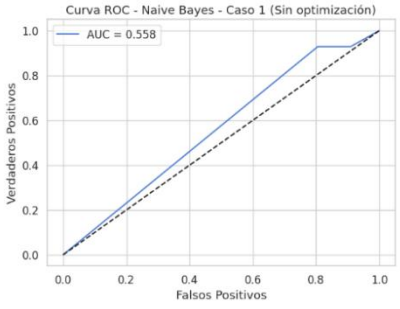
El análisis del modelo Naive Bayes reveló distinciones muy evidentes entre su versión original y sus versiones afinadas. En el Caso 1, sin ninguna clase de modificaciones, el modelo mostró un desempeño deficiente, con una precisión de escasamente 0.3143 y un AUC de 0.558, valores que demuestran una capacidad predictiva casi igual a la casualidad. Su mayor problema estuvo en la especificidad, cayendo hasta 0.0909 y produjo 140 falsos positivos, revelando una notable incapacidad para identificar a las personas sin autismo. No obstante, los Casos 2 y 3, donde se usó primero la optimización de hiperparámetros y luego GridSearchCV con validación cruzada, obtuvieron mejoras espectaculares, llegando a desempeños excelentes. La precisión se disparó hasta casi 0.97, el AUC alcanzó 0.997 y la especificidad sobrepasó 0.96, disminuyendo los falsos positivos a cantidades mínimas como 4 y 6. Los dos métodos originaron resultados casi idénticos, mostrando que la optimización de parámetros, más que la validación cruzada por si sola, fue el componente crucial que transformó al modelo en una herramienta altamente efectiva.

```
========== CASO 1 ==========
Modelo Naive Bayes SIN optimización de parámetros

============================================================
RESULTADOS DEL MODELO: Naive Bayes - Caso 1 (Sin optimización)
============================================================
Matriz de Confusión:
------------------------------------------------------------
              Predicción
              Neg     |    Pos
Real Neg  |    14    |    140
Real Pos  |     4    |     52

VN:   14 | FP:  140
FN:    4 | VP:   52
------------------------------------------------------------
Métricas derivadas:
------------------------------------------------------------
Accuracy (Exactitud):          0.3143
Sensibilidad (Recall +):       0.9286
Especificidad (Recall -):      0.0909
VPP (Precisión Positiva):      0.2708
VPN (Precisión Negativa):      0.7778
------------------------------------------------------------
Reporte de Clasificación:
------------------------------------------------------------
              precision    recall  f1-score   support

NO Autismo       0.78      0.09      0.16       154
SÍ Autismo       0.27      0.93      0.42        56
```



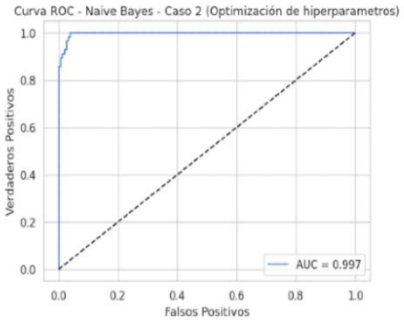

```
========== CASO 2 ==========
Optimización de hiperparámetros

Mejor var_smoothing encontrado: 1.0
Accuracy obtenido en test: 0.9714

============================================================
RESULTADOS DEL MODELO: Naive Bayes - Caso 2 (Optimización de hiperparametros)
============================================================
Matriz de Confusión:
------------------------------------------------------------
              Predicción
              Neg     |    Pos
Real Neg  |   150    |     4
Real Pos  |     2    |     54

VN: 150 | FP:   4
FN:   2 | VP:  54
------------------------------------------------------------
Métricas derivadas:
------------------------------------------------------------
Accuracy (Exactitud):          0.9714
Sensibilidad (Recall +):       0.9643
Especificidad (Recall -):      0.9740
VPP (Precisión Positiva):      0.9310
VPN (Precisión Negativa):      0.9868
------------------------------------------------------------
Reporte de Clasificación:
------------------------------------------------------------
              precision    recall  f1-score   support

NO Autismo       0.99      0.97      0.98       154
SÍ Autismo       0.93      0.96      0.95        56
```



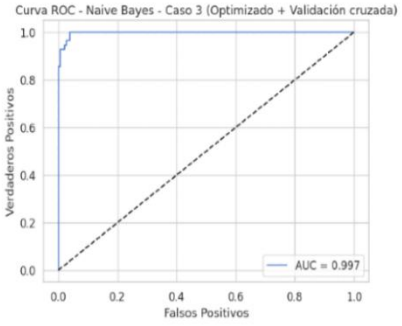

```
========== CASO 3 ==========
Optimización + Validación Cruzada (GridSearchCV)

Fitting 5 folds for each of 40 candidates, totalling 200 fits

============================================================
RESULTADOS DEL MODELO: Naive Bayes - Caso 3 (Optimizado + Validación cruzada)
============================================================
Matriz de Confusión:
------------------------------------------------------------
              Predicción
              Neg     |    Pos
Real Neg  |   148    |     6
Real Pos  |     1    |     55

VN: 148 | FP:   6
FN:   1 | VP:  55
------------------------------------------------------------
Métricas derivadas:
------------------------------------------------------------
Accuracy (Exactitud):          0.9667
Sensibilidad (Recall +):       0.9821
Especificidad (Recall -):      0.9610
VPP (Precisión Positiva):      0.9016
VPN (Precisión Negativa):      0.9933
------------------------------------------------------------
Reporte de Clasificación:
------------------------------------------------------------
              precision    recall  f1-score   support

NO Autismo       0.99      0.96      0.98       154
SÍ Autismo       0.90      0.98      0.94        56
```



3.2 Métricas SVM

Los resultados obtenidos de cada uno de los casos utilizando el algoritmo SVM se observan sin muchas diferencias, teniendo una precisión de 92.86% para el caso sin optimización, siendo el único en el que se observan diferencias frente a los otros dos casos optimizados. El caso 2 y 3 obtienen una precisión de 94.29%, superando por poco el caso 1. La optimización de hiperparámetros se ve reflejada en el aumento, aunque la validación cruzada no demuestra un cambio, por lo que se nota un fortalecimiento al modelo tan sólo utilizando una optimización de parámetros.

```
============================================================
Caso 1 (Sin Optimización)
============================================================

Matriz de Confusión (Filas reales, Columnas predichas):
[[101   2]
 [  8  29]]
VP: 29, VN: 101, FP: 2, FN: 8

Métricas:
                Valor
VP          29.000000
VN         101.000000
FP           2.000000
FN           8.000000
Accuracy     0.928571
Sensibilidad 0.783784
Especificidad 0.980583
VPP          0.935484
VPN          0.926606

Reporte de Clasificación:
              precision    recall  f1-score   support

          NO       0.93      0.98      0.95       103
         YES       0.94      0.78      0.85        37

    accuracy                           0.93       140
   macro avg       0.93      0.88      0.90       140
weighted avg       0.93      0.93      0.93       140
```

```
============================================================
Caso 2 (Con optimización cruzada)
============================================================

Matriz de Confusión (Filas reales, Columnas predichas):
[[101   2]
 [  6  31]]
VP: 31, VN: 101, FP: 2, FN: 6

Métricas:
                Valor
VP          31.000000
VN         101.000000
FP           2.000000
FN           6.000000
Accuracy     0.942857
Sensibilidad 0.837838
Especificidad 0.980583
VPP          0.939394
VPN          0.943925

Reporte de Clasificación:
              precision    recall  f1-score   support

          NO       0.94      0.98      0.96       103
         YES       0.94      0.84      0.89        37

    accuracy                           0.94       140
   macro avg       0.94      0.91      0.92       140
weighted avg       0.94      0.94      0.94       140
```

```
============================================================
Caso 3: Optimización de Hiperparámetros con Validación Cruzada
============================================================

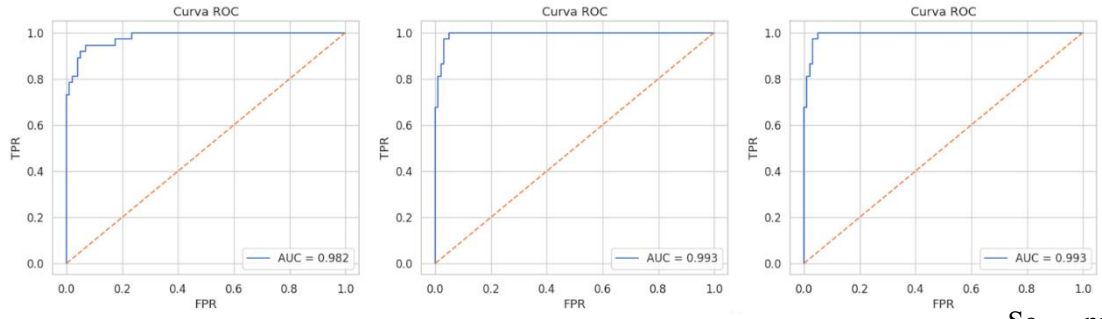
Matriz de Confusión (Filas reales, Columnas predichas):
[[101   2]
 [  6  31]]
VP: 31, VN: 101, FP: 2, FN: 6

Métricas:
                Valor
VP          31.000000
VN         101.000000
FP           2.000000
FN           6.000000
Accuracy     0.942857
Sensibilidad 0.837838
Especificidad 0.980583
VPP          0.939394
VPN          0.943925

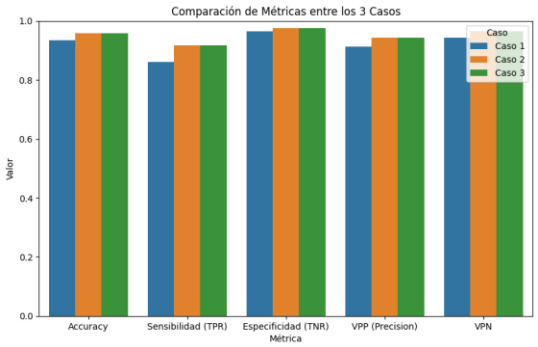
Reporte de Clasificación:
              precision    recall  f1-score   support

          NO       0.94      0.98      0.96       103
         YES       0.94      0.84      0.89        37

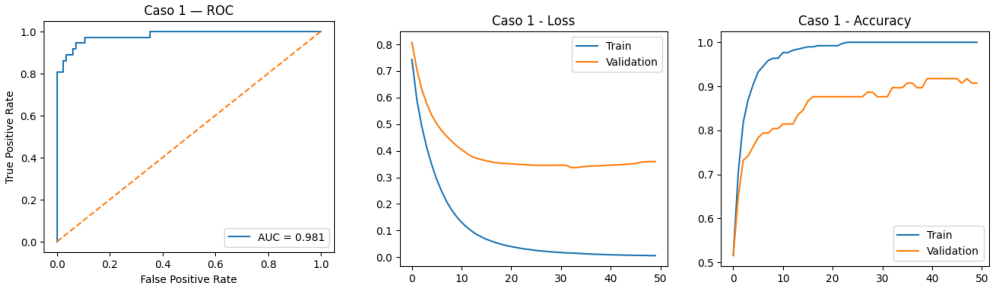
    accuracy                           0.94       140
   macro avg       0.94      0.91      0.92       140
weighted avg       0.94      0.94      0.94       140
```



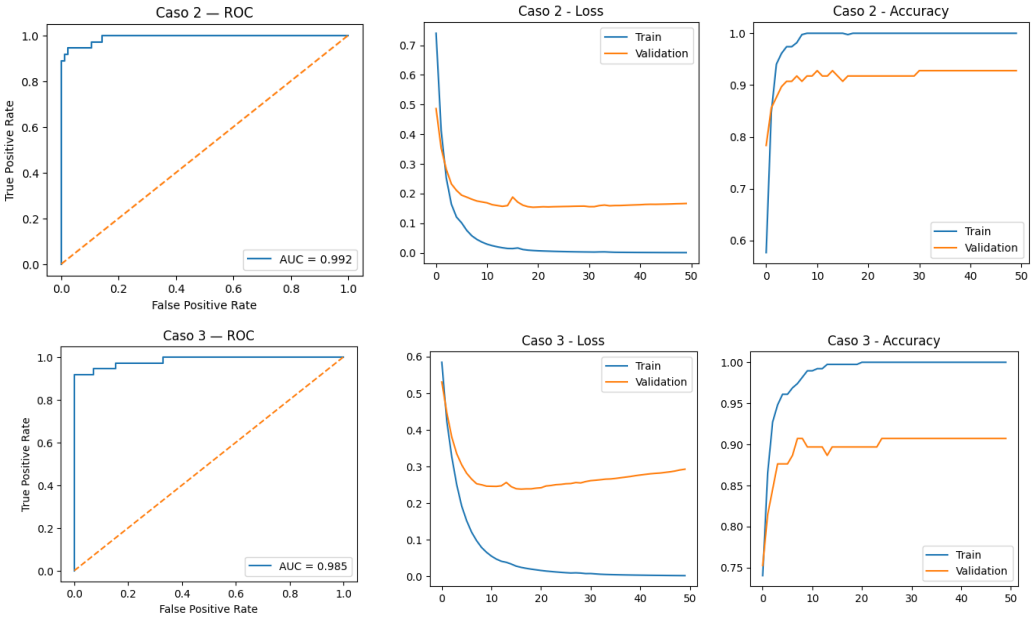
3.3 Métricas Redes Neuronales




```
Accuracy 1: 0.9338842975206612
Accuracy 2: 0.9586776859504132
Accuracy 3: 0.9586776859504132
Sensibilidad 1: 0.8611111111111112
Sensibilidad 2: 0.9166666666666666
Sensibilidad 3: 0.9166666666666666
Especificidad 1: 0.9647058823529412
Especificidad 2: 0.9764705882352941
Especificidad 3: 0.9764705882352941
VPP 1: 0.9117647058823529
VPP 2: 0.9428571428571428
VPP 3: 0.9428571428571428
VPN 1: 0.9425287356321839
VPN 2: 0.9651162790697675
VPN 3: 0.9651162790697675
```



Se presentan métricas visiblemente superiores tanto en el caso 2 como en el caso 3, pero dado que el caso 1 solo se centra en el split 80/20, estos datos no son tan confiables y tienen una alta probabilidad de que en una siguiente iteración su valor cambie drásticamente a uno mejor o peor. De las gráficas Loss-Accuracy se desprende que el modelo 2 generaliza y aprende muy bien, en los casos 1 y 3 hay un aprendizaje excelente, pero sin tendencias a mejorar mucho como se desprende en el grafico Loss de ambos, el Caso 1 tiene buena generalización y el caso 3 tiene un sobreajuste alto, por lo tanto, no generaliza bien. En el caso de la sensibilidad, los 3 modelos demuestran ser muy sensibles al momento de separar las clases, lo que se puede deducir en base a las curvas ROC de los 3 modelos, a la par de los niveles de sensibilidad arrojados más arriba.



4. Comparación entre algoritmos

Dados los resultados anteriores, se desprende que:

- Mayor AUC, esto indica la mejor habilidad para distinguir entre clases.
- Mayor Exactitud es un indicador de su rendimiento general.
- Mejor Equilibrio, alto desempeño en Sensibilidad y Especificidad, muy importante.

Ganador, el modelo Naive Bayes - Caso 2, sobresale: Naive Bayes Caso 2 (Optimizando Hiperparámetros) sigue en la cima, por varias razones.

- AUC máximo (0.997), empatado con el Caso 3; esto es lo más alto que vimos, mostrando una casi perfecta diferenciación de clases (Autismo vs. No Autismo).
- Exactitud máxima (0.9714), este valor, es la exactitud más alta que vimos en todos los casos, increíble.
- Mejor Sensibilidad (0.9643) tiene la mejor habilidad para detectar correctamente la condición positiva (Sí Autismo), esencial para diagnósticos correctos.

Mención honorífica a Naive Bayes - Caso 3, un fuerte competidor: El Caso 3 de Naive Bayes es una alternativa formidable, especialmente si el control de Falsos Positivos es prioritario.

- Especificidad máxima (0.9933), es lo mejor identificando correctamente casos negativos (No Autismo).
- VPP máximo. (0.9886) Dice, el modelo suele acertar cuando predice "Si Autismo", casi siempre es correcto (alta precisión).

Conclusión: El modelo Naive Bayes - Caso 2 (Optimización de hiperparámetros), por lograr el mayor Accuracy (0.9714), la mejor Sensibilidad (0.9643), y el AUC máximo (0.997) de toda la comparación, es el modelo más perfecto.

Conclusiones

El machine learning es una herramienta muy importante para el procesamiento y el análisis de datos, habiendo sido demostradas sus capacidades dentro de las pruebas realizadas con los diversos métodos de entrenamientos aplicados en el caso de estudio elegido, donde en su generalidad se demostró que es posible mejorar aún más su utilidad y efectividad, además de ayudar a discernir qué método es mejor en diferentes posibles escenarios, destacando las Redes Neuronales por sobre los otros como un modelo estable, fiable y óptimo.

Bibliografía

- UCI machine learning repository. (s/f). Uci.edu. Recuperado el 3 de diciembre de 2025, de https://archive.ics.uci.edu/dataset/426/autism+screening+adult
- Santana, E., & Perfil, V. T. mi. (s/f). Red Neuronal con dataset iris. Blogspot.com. Recuperado el 3 de diciembre de 2025, de https://apuntes-r.blogspot.com/2015/11/red-neuronal-con-dataset-iris.html
- Naive Bayes classifiers. (2017, marzo 3). GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/
- Support vector machines. (s/f). Scikit-Learn. Recuperado el 3 de diciembre de 2025, de https://scikit-learn.org/stable/modules/svm.html
- ¿Qué es una máquina de vectores soporte? (2025, enero 29). Ibm.com. https://www.ibm.com/es-es/think/topics/support-vector-machine
- Neural networks. (s/f). Mathworks.com. Recuperado el 3 de diciembre de 2025, de https://la.mathworks.com/help/stats/neural-networks-for-classification.html